Relatório de entrega — Bibliotequinha da Rhay 2.0
Autora: Rhayanna | Revisão para entrega em 04/11/2026

Bibliotequinha da Rhay 2.0

O que é a Bibliotequinha

Desenvolvi a Bibliotequinha para organizar uma biblioteca e controlar a retirada e a devolução dos livros. Cada cadastro representa um exemplar físico, com código próprio. Dois exemplares podem ter o mesmo título, mas não o mesmo código.

Sou Rhayanna e realizei o projeto com assistência técnica do Codex na implementação, nos testes e na documentação. O professor está ciente da realização individual, conforme minha confirmação; não alego autorização formal escrita. A entrega ao GitHub foi autorizada em 04/11/2026: repositório do projeto (https://github.com/Rh4yanna/bibliotequinha-da-rhay).

Tecnologias utilizadas

Uso Laravel 12/PHP 8.2 na aplicação, Blade nas telas e Eloquent para consultar tabelas por classes. PostgreSQL guarda os dados, hospedados no Neon. Composer instala as dependências PHP; Node.js/npm e Vite preparam o visual; PHPUnit testa e Git registra a evolução.

No MVC, Model representa dados, Controller recebe requisições e View mostra a tela. Services reúnem operações, Form Requests validam dados e Policies conferem permissões. Exemplo: LoanController chama LoanService para registrar a retirada.

O que o sistema faz

- Cadastra, lista, mostra, edita e exclui livros. Essas operações formam o CRUD.
- Organiza categorias e pesquisa livros por título, autor, código e situação.
- Registra retirada, prazo e devolução, mantendo o histórico.
- Permite até três empréstimos ativos por pessoa e um por exemplar.
- Oferece API de livros: acesso por HTTP com dados em JSON.

Perfil	Permissões principais
admin	Gerencia livros/categorias, registra/devolve e exclui livro sem histórico
bibliotecario	Cadastra/edita livros e registra/devolve empréstimos
consulta	Lê o catálogo e apenas os próprios empréstimos

O cadastro público cria consulta. Categoria com livros, livro/conta com histórico não podem ser excluídos. Devolver mantém a retirada. Não há reservas, multas ou tela de auditoria; autor é texto. Auditoria é o registro automático das mudanças.

O que foi reaproveitado do primeiro bimestre

Confirmei que a Bibliotequinha foi o projeto entregue no primeiro bimestre. O commit dcfd862, de 16/09/2026, registra a versão anterior; essa data é do Git, não uma data de entrega ao professor.

Reaproveitei users, categories e books, autenticação Breeze, os três perfis, CRUD, busca e painel. A migration 2026_09_13_000001_create_library_tables.php já criava categories/books e a ligação category_id.

Na versão 2.0, 2026_11_02_000001_create_loans_and_api_tokens.php acrescentou loans (empréstimos), loan_audits (histórico automático) e api_tokens (acesso à API). 2026_11_02_000002_install_postgres_library_objects.php instala as novas rotinas SQL e índices. O catálogo anterior foi preservado. Exemplares antigos marcados como emprestados sem tomador precisam de conferência manual, sem inventar histórico.

Continuei esse banco para aplicar regras úteis de disponibilidade, prazo e devolução. Em 02/11/2026, foram implementados e verificados empréstimos, API, SQL e Neon. Em 03/11/2026, consolidei a documentação sem alterar dados ou regras.

Como instalar e executar

Para uma cópia nova, instale PHP 8.2 ou superior com pdo_pgsql (driver PostgreSQL), Composer, Node.js/npm e PostgreSQL. Os scripts locais foram validados com PostgreSQL 18. Entre na pasta e execute em PowerShell:

```
composer install
Copy-Item .env.example .env
php artisan key:generate
npm ci
npm run build
```

Não copie o exemplo por cima do .env já configurado deste projeto. Crie um banco local vazio chamado biblioteca e configure a conexão conforme a próxima seção. Depois de conferir o destino local:

```
php artisan migrate --seed
php artisan serve --host=127.0.0.1 --port=8001
```

migrate cria a estrutura pelas migrations (arquivos de alteração do banco); --seed insere exemplos. Abra http://127.0.0.1:8001. Use npm run dev para editar o visual e npm run build para prepará-lo.

DemoUserSeeder define contas admin@biblioteca.test, bibliotecario@biblioteca.test e consulta@biblioteca.test e sua senha de estudo. Não use publicamente. O seeder não redefine contas/livros existentes. E-mails de desenvolvimento ficam em storage/logs/laravel.log.

Como configurar o PostgreSQL e o Neon

Exemplo para o banco local novo; os valores em maiúsculas são espaços para sua configuração privada:

```
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
```

DB_PORT=5432
DB_DATABASE=biblioteca
DB_USERNAME=SEU_USUARIO_LOCAL
DB_PASSWORD=SUA_SENHA_LOCAL
DB_SSLMODE=disable

Se usar DB_URL, ela define a conexão completa: não deixe uma URL Neon quando pretende usar local. Nesta máquina Windows, para habilitar o driver no terminal atual:

$env:PHP_INI_SCAN_DIR = Join-Path $PWD 'scripts/php-conf'

O Neon principal usa neondb no projeto bibliotequinha-rhay e sslmode=require, proteção obrigatória na comunicação. A conexão privada fica no .env; .env.neon também contém dados privados. .env.local preserva a configuração local anterior. Nunca envie esses arquivos ou tokens.

A URL usa pooler (intermediário de conexões); backup/transferência usaram host direto. NeonPostgresConnector acrescenta a identificação de endpoint necessária à biblioteca antiga libpq 11.4 do PHP. Isso só se aplica a hosts Neon.

Em banco existente, faça backup e confira o destino antes de php artisan migrate. Não use migrate:fresh, db:wipe ou restauração por cima do banco principal: podem apagar dados. Não rode seeders de demonstração no Neon.

Como utilizar a aplicação

Faça login e abra Livros para cadastrar, consultar, editar ou excluir conforme sua permissão. Em Empréstimos, escolha Registrar empréstimo, leitor, exemplar e prazo futuro. Para devolver, abra Ver empréstimo e clique Registrar devolução. A contagem da tela é da conta logada.

Rota web	Uso
/login; /register	Entrar; criar conta consulta
/dashboard	Indicadores
/livros; /livros/{id}	Catálogo e ficha; POST cria, PUT/PATCH edita, DELETE exclui
/categorias	Categorias, com escrita restrita ao admin
/emprestimos; /emprestimos/{id}	Retirada, consulta e devolução
/profile	Própria conta

routes/web.php e routes/auth.php têm a lista completa. O servidor confere permissões, e os formulários usam CSRF, proteção contra envio indevido de outra página.

Para gravar, use o banco fictício local e .env.gravacao, sem trocar o .env principal:

php scripts/console-gravacao.php destino
php artisan serve --env=gravacao --host=127.0.0.1 --port=8002 --no-reload

O PostgreSQL local da porta 55433 precisa estar iniciado. Em ambiente novo, scripts/Preparar-Gravacao.ps1 cria outro banco vazio; não sobrescreve uma configuração de gravação existente.

Cadastre apresentacao@example.test com senha privada. O apoio console-gravacao.php promover torna essa conta admin só localmente; preparar cria VIDEO-EMPR-01; auditoria consulta seu histórico.

Para os dez minutos do professor: objetivo/origem (1 min), CRUD (1,5), MVC/validação (0,5), contagem/retirada/auditoria/devolução (2,5), API (1), índices/Neon/backup (1,5), testes/conclusão (2). Empreste para a própria conta para mostrar 0 ? 1 ? 0. Comprove auditoria por consulta, pois não existe essa tela. Não grave senhas/tokens; trabalho individual confirmado não altera automaticamente o limite de tempo.

Rotas principais da API e exemplos

A API exige Bearer, uma chave enviada no cabeçalho. O banco guarda seu hash (representação calculada) e validade. php artisan api:token admin@biblioteca.test imprime um token de sete dias: execute em privado. --revoke revoga os tokens da conta. Não há emissão pública; o limite é 60 chamadas/minuto.

Método / caminho	Sucesso
GET /api/livros; GET /api/livros/{id}	200
POST /api/livros	201
PUT/PATCH /api/livros/{id}	200
DELETE /api/livros/{id}	204, sem corpo

Todos consultam; equipe escreve; só admin exclui livro sem histórico. PUT exige dados completos; PATCH aceita só os alterados. BookResource define os campos JSON; a lista tem data, links e meta, informações de paginação.

Exemplo PowerShell no servidor local, com token privado em BIBLIOTECA_TOKEN e category_id ajustado para uma categoria existente:

$cabecalhos = @{ Authorization = "Bearer $env:BIBLIOTECA_TOKEN"; Accept = 'application/json' }
Invoke-RestMethod http://127.0.0.1:8002/api/livros -Headers $cabecalhos
$corpo = @{ title='Livro fictício'; author='Autora Fictícia'; code='API-EXEMPLO-01'; publication_year=2020; category_id=1; status='disponivel' } | ConvertTo-Json
Invoke-RestMethod http://127.0.0.1:8002/api/livros -Method Post -Headers $cabecalhos -ContentType 'application/json' -Body $corpo

Não imprima cabeçalhos. Código deve ser único; título, autor, ano, categoria e situação são obrigatórios. publisher/synopsis são opcionais. O estado emprestado vem da retirada, não do cadastro manual. Erros: 401 sem autenticação, 403 proibido, 404 ausente, 422 inválido e 429 excesso de chamadas.

Para o vídeo, php scripts/console-gravacao.php api demonstra GET/POST locais sem imprimir o token temporário. BookApiTest cobre o CRUD completo.

Functions, procedures, triggers e índices

As definições reais estão em database/sql/01_library_objects.sql

(database/sql/01_library_objects.sql).

Objeto	O que faz
fn_quantidade_emprestimos_ativos	Recebe usuário e retorna número de retiradas sem devolução
fn_livro_disponivel	Recebe exemplar e retorna verdadeiro/falso
sp_registrar_emprestimo	Registra retirada com usuário, livro e prazo
sp_registrar_devolucao	Preenche retorno do empréstimo ativo
loans_validate	Valida antes de gravar/excluir empréstimos
loans_audit	Registra mudança e sincroniza situação do livro
books_protect_status	Impede situação incompatível com retirada ativa
books_title_order_idx	Ajuda a ordenar por título/ID
loans_active_user_idx	Índice só dos ativos, por usuário/prazo
loans_one_active_copy_idx	Impede dois ativos do mesmo exemplar

Function devolve um valor com SELECT; procedure executa uma operação com CALL; trigger dispara automaticamente quando a tabela muda. As funções auxiliares das triggers não substituem as duas functions de negócio. Os dois primeiros índices atendem consultas; o terceiro protege a regra de unicidade. Índice não garante ganho em toda busca, especialmente %termo% e tabelas pequenas.

LoanController recebe o formulário, LoanRequest/LoanPolicy conferem entrada e permissão e LoanService executa SELECT/CALL com valores separados do SQL. Uma transação confirma tudo junto ou desfaz se houver erro. Bloqueios temporários das linhas ajudam a tratar operações simultâneas. Devolver gera UPDATE na auditoria, sem apagar o INSERT da retirada.

Para reconstruir por SQL, escolha banco local novo e vazio: aplique 00_estrutura.sql e 03_dados_ficticios.sql com psql e -v ON_ERROR_STOP=1 -1; depois rode 04_validacao_local.sql. O arquivo 00 já inclui os objetos do 01. Não misture essa reconstrução com migrations em banco preenchido. 02_demonstracao.sql traz consultas e EXPLAIN, que mostra o plano escolhido pelo PostgreSQL.

Como executar os testes

powershell -NoProfile -ExecutionPolicy Bypass -File scripts/Test-Postgres.ps1

O script usa PostgreSQL 18 em bibliotequinha_test/127.0.0.1:55433, com .env.testing privado, e recusa destino inesperado. Se necessário, informe -PostgresBin. Esse banco separado pode ser reconstruído: nunca aponte PHPUnit para Neon, campus ou o banco da gravação. Não rode duas suítes ao mesmo tempo, pois compartilham cache de telas.

Em tests/Feature, LibraryTest cobre páginas/CRUD, BookApiTest cobre JSON/status e PostgresLoanTest cobre empréstimos/auditoria. Os dois primeiros têm os dois testes web e dois API exigidos. tests/Unit/NeonPostgresConnectorTest.php verifica a conexão adaptada.

php artisan test usa SQLite em memória e pula os testes PostgreSQL; não comprova suas procedures/triggers. A opção -Entrega do script confere também SQL e backup fictício. Os logs abaixo são anteriores à limpeza.

Como realizar backup e restauração

Backup é a cópia para recuperar dados. Restaure somente em banco novo vazio, com ferramentas compatíveis. Exemplo manual, preenchendo os nomes localmente e digitando a senha no pedido de -W:

pg_dump -h 127.0.0.1 -p PORTA_LOCAL -U USUARIO_LOCAL -W -d BANCO_LOCAL -Fc -f biblioteca.dump
createdb -h 127.0.0.1 -p PORTA_LOCAL -U USUARIO_LOCAL -W biblioteca_restore_test
pg_restore -h 127.0.0.1 -p PORTA_LOCAL -U USUARIO_LOCAL -W -d biblioteca_restore_test --no-owner --no-acl --exit-on-error --single-transaction biblioteca.dump

-Fc gera o formato customizado para pg_restore. Confira contagens, conteúdo e execução das rotinas depois de restaurar. O backup já inclui o SQL especial. Não use --clean ou restauração sobre o banco principal.

powershell -NoProfile -ExecutionPolicy Bypass -File scripts/Test-Postgres.ps1 -Entrega automatiza testes, reconstrução SQL e restauração fictícia só em bancos locais separados. O dump fica em storage/app/entrega-local/backup-ficticio.dump, acompanhado de .sha256, assinatura que ajuda a conferir a integridade. Backups reais continuam privados em storage/app/backups e não entram na entrega.

Resultados reais dos testes e validações

Em 04/11/2026, a validação de entrega passou com 46 testes e 248 verificações, em 1 minuto e 27,876 segundos. Assertion é uma comparação do resultado esperado. A reconstrução/restauração fictícia teve 3 usuários, 4 categorias, 16 livros, 2 empréstimos e 3 auditorias, com conteúdo igual conferido por assinatura. As regras foram testadas no destino e as operações de conferência desfeitas com ROLLBACK.

Na transferência Neon anterior autorizada, foram comparados 7 usuários, 16 livros, 4 categorias e 10 migrations, com loans/loan_audits vazios. As rotinas e índices foram confirmados; retirada/devolução Laravel funcionaram em transação revertida, sem dados de teste persistidos. O banco local foi preservado.

Evidência	Conteúdo
finalizacao-local.txt (docs/evidencias/finalizacao-local.txt)	46/248, SQL e restauração fictícia
postgres-validacao.txt (docs/evidencias/postgres-validacao.txt)	Etapa anterior: 45/243, antes do teste do conector
neon-transferencia.txt (docs/evidencias/neon-transferencia.txt)	Backups e comparação anteriores
neon-validacao.txt (docs/evidencias/neon-validacao.txt)	Operações anteriores no Neon
neon-testes-isolados.txt (docs/evidencias/neon-testes-isolados.txt)	Testes locais após ajuste do conector

Também foram conferidos build, 43 rotas e login/registro 200 e API sem token 401 no vídeo local. Não houve teste de carga ou revisão visual completa. A limpeza de documentação anterior

não reexecutou testes. Para a entrega em 04/11/2026, a suíte e a restauração fictícia foram repetidas apenas localmente, sem acessar o Neon.

O relatório PDF (docs/RELATORIO_ENTREGA.pdf) foi mantido por ser exigido pelo professor, assim como SQL, testes e evidências. O enunciado pede a stack adotada, dois índices/functions/procedures/triggers, CRUD web/API, integração SQL/Laravel, dois testes de cada tipo e backup/restauração. A implementação tem esses itens; ainda preciso ensaiar e demonstrar domínio. A nota se divide em um ponto para Banco de Dados II, um para Back-end e um para apresentação/arguição. A data da capa, 12/10/2026, não foi tratada como prazo.

Este é o único Markdown do projeto. O PDF, os arquivos técnicos e os backups privados foram preservados. O envio da versão final ao GitHub foi autorizado em 04/11/2026. A limpeza de documentação anterior não alterou o banco; não há deploy da aplicação.